

SW 마일리지 토큰화 프로젝트

2025 블록체인 프로젝트 7조

| 목차

- 기존 SW 마일리지 및 토큰화 프로젝트 소개
- 데모
- 아키텍처 및 구현 설명
- 회고

| 소프트웨어 마일리지?

SW MILEAGE

2024 SW 마일리지 프로그램

신청 기간
2024.11.1 - 12.6



| 신청 대상

- 소프트웨어융합대학 소속 재학생 및 다전공 재학생 (8학기 이내)

| 신청 방법 및 기간

- 신청방법: SW중심대학 홈페이지 - [학 생지원] - [SW마일리지 신청] 페이지에서 마일리지 신청 (<https://swedu.khu.ac.kr/06/05.php>)
- 신청기간: 2024년 11월 1일 - 12월 6일

| 결과발표

- 2024년 12월 9일 예정

| 주의사항

- 마일리지 신청 후 반드시 증명자료 업로드 (증빙내용에 대한 신청 x)
- 사업단에서 운영한 프로그램에서 입상 하여 상금을 수상한 학생들에게는 마일리지 점수를 부여하지 않음
- 사업단에서 운영한 프로그램이 아니라도, 같은 내용으로 타 행사에 참여하여 수상한 경우 마일리지 점수를 부여하지 않음

| 문의사항

- SW중심대학 사업단 연구교수 허림 (rhaw@khu.ac.kr / 031-201-2961)


경희대학교
SW중심대학사업단



마일리지 취득 배점에 따라 등급별 장학금 지급

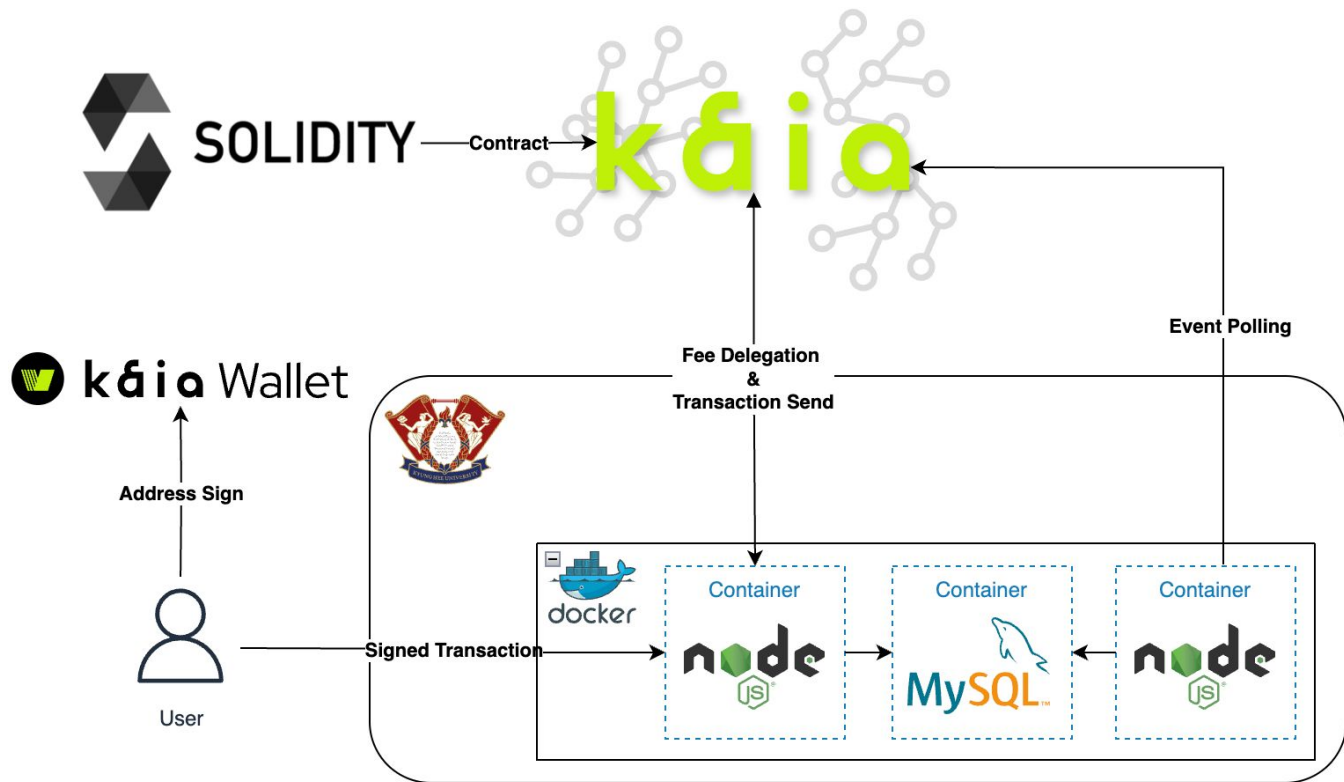
구분	지급기준	장학금	비고
1등급	마일리지 취득 상위 10%	100만원	
2등급	마일리지 취득 상위 30%	70만원	
3등급	마일리지 취득 상위 50%	50만원	
4등급	마일리지 취득 상위 70%	30만원	
5등급	마일리지 100점 이상 취득자 전원	20만원	상위등급과 중복 배제

마일리지 배점표

분야	비교과활동	배점
국내 학술회의 논문 발표 (사사문구 필요)	주저자 (100%)	50
	공동저자 (50%)	25
국내 학술지 논문 발표	주저자 (100%)	100

기능 소개

| Architecture



| Minimal Proxy

- 다음 학기가 된다면?

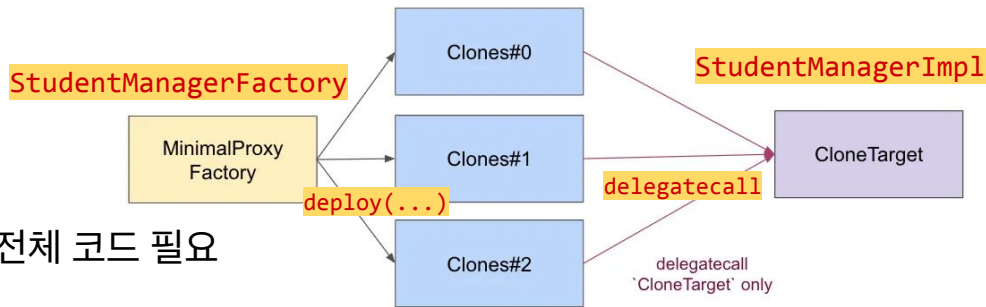
- 다음 학기 관리자는 재배포 위해 컨트랙트 전체 코드 필요
- 똑같은 컨트랙트 코드를 재배포 하게됨

- Minimal Proxy

- (사전 지식) 컨트랙트 실행 과정 중 새로운 컨트랙트를 배포할 수 있음 (Factory 컨트랙트)
- ...Impl 구현체 컨트랙트 배포
- 구현체 컨트랙트에 delegatecall만 하는 가벼운 컨트랙트를 Factory로 배포
- 매번 전체 컨트랙트 코드를 갖고있지 않아도 가볍게 배포 가능

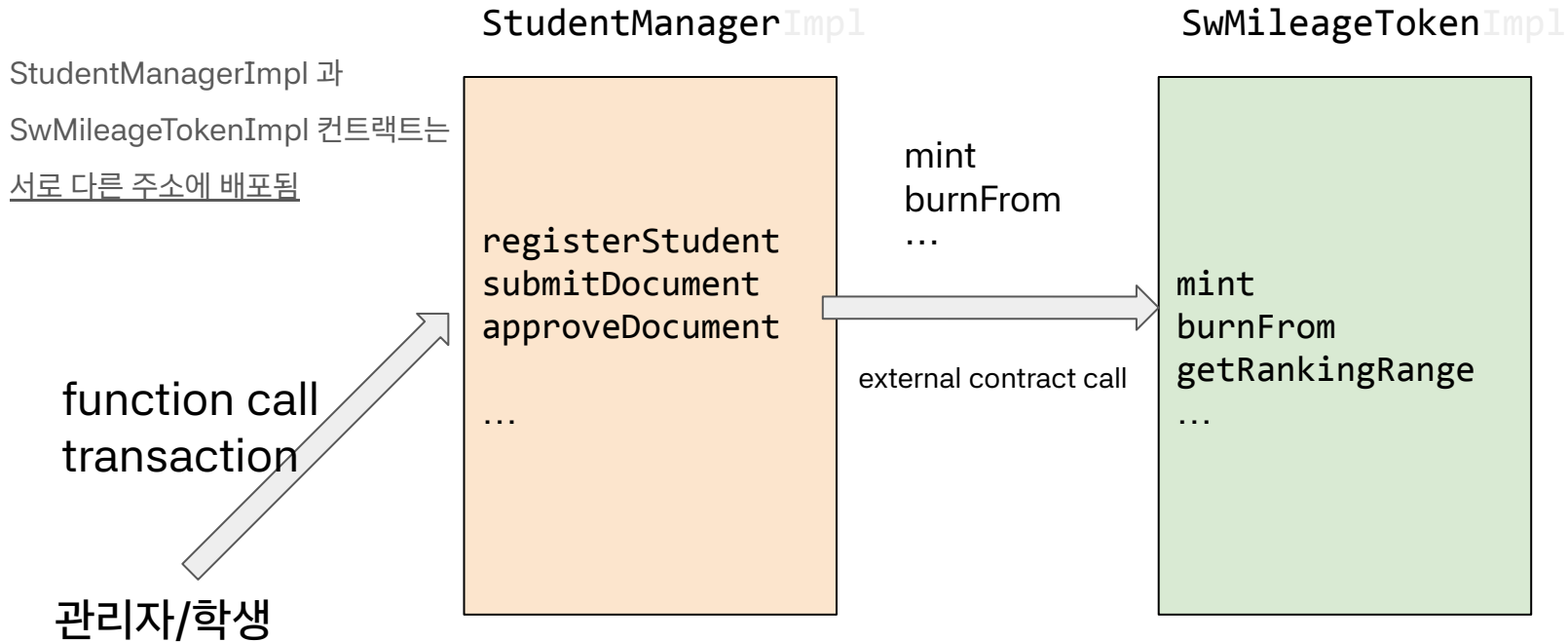
- 이후 발표자료에서는 편의상 컨트랙트 이름에 Impl를 붙이지 않는 경우도 혼용

- OpenZeppelin Clones 라이브러리 사용 (openzeppelin-contracts/contracts/proxy/Clones.sol)



source: <https://medium.com/@aiden.p/업그레이더블-컨트랙트-써-리즈-part-4-미니멀-프록시-컨트랙트-해체-분석하기-7cf518133ba0>

| Smart Contract



*proxy pattern 과정 생략한 그림

| Smart Contract

- **SwMileageTokenImpl**
 - KIP7(ERC20), 마일리지 홀더 랭킹 기능
 - 1마일리지 = 1토큰
- **StudentManagerImpl**
 - 학생 계정 관리
 - 학생 등록, 마일리지 신청, 지급, 학생 지갑 변경, etc
 - 사실상 토큰을 제외한 모든 중요 로직을 실행
- **SwMileageTokenFactory**
- **StudentManagerFactory**

| StudentManager 컨트랙트

- **registerStudent(bytes32 stuentId)**
 - 컨트랙트에 (msg.sender, studentId (해시)) 등록
 - studentId는 학생 정보를 해시한 것
- **submitDocument(bytes32 docHash)**
 - 학생이 제출된 자료 해시(원본 문서는 웹으로)와 함께 마일리지 지급 요청
 - 제출 문서 해시는 일종의 체크섬으로 블록체인에 저장
- **approveDocument(uint256 documentIndex, uint256 amount, bytes32 reasonHash)**
 - 어드민이 제출된 자료중 인덱스에 해당하는 마일리지 지급 요청 승인/반려

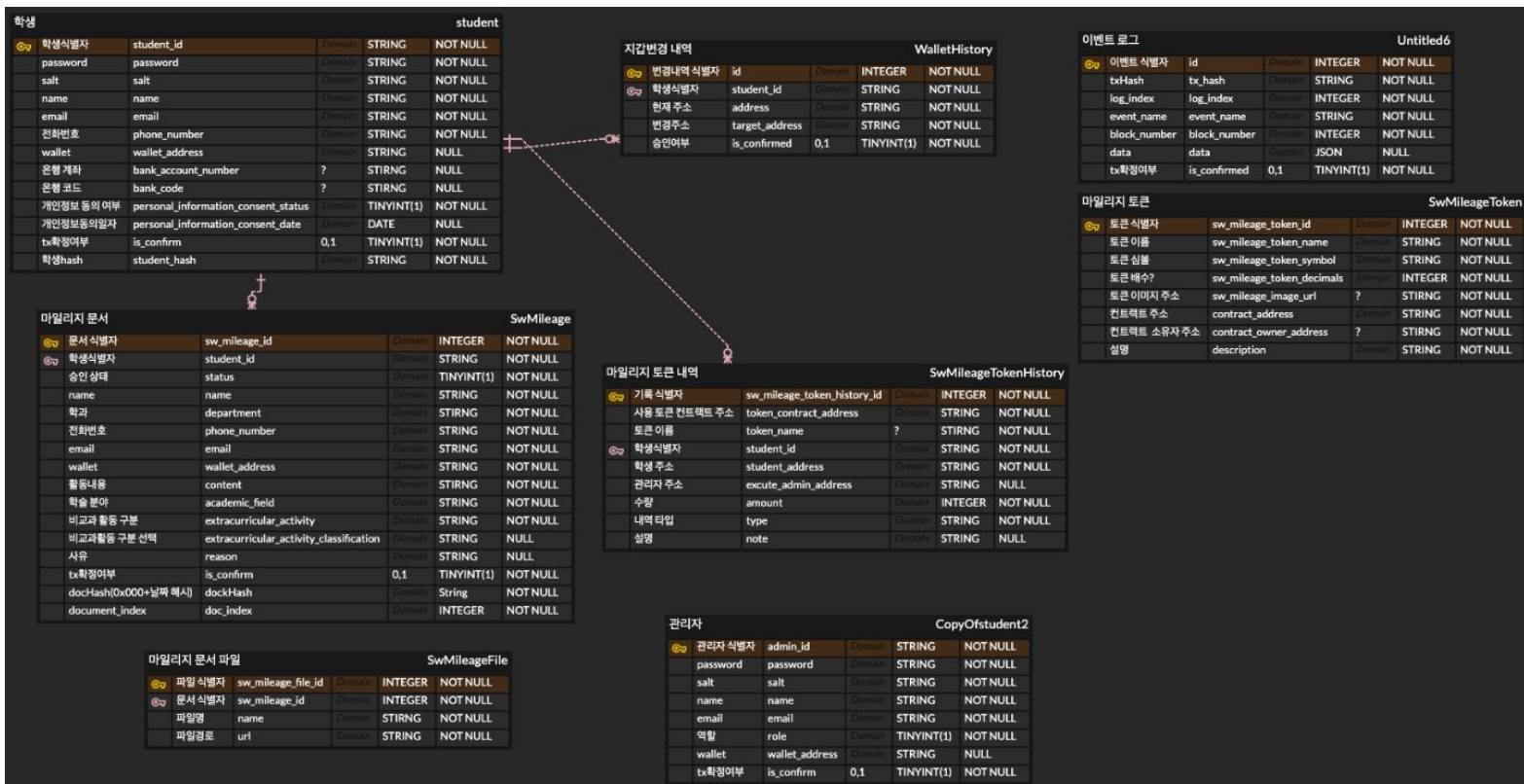
| StudentManager 컨트랙트

- **proposeAccountChange(address targetAccount)**
 - 학생이 자신이 사용중인 지갑 주소(studentId에 매핑된 지갑 주소)를 targetAccount로 변경할 것임을 등록.
- **confirmAccountChange(bytes32 studentId)**
 - studentId 학생의 주소 변경 요청(targetAccount)이 msg.sender와 같을 경우 studentId에 해당하는 학생의 지갑 주소를 msg.sender로 바꿈
- 기존 사용 지갑으로 targetAccount에게 변경 요청을 보내고
→ targetAccount가 이를 승인하는 형태라고 볼 수 있음
- 기존 사용하던 지갑 으로 proposeAccountChange -> 변경할 지갑으로 confirmAccountChange 호출하여 2번에 걸쳐 변경

| 개발중 이슈

- 변경할 주소(목적지)에 대한 소유권 증명
 - 최초 가입시 별도의 신원 인증(khu 계정) 과정이 없기 때문에 더미 계정 생성 가능
 - 단순히 인자에 넣은 주소로 즉시 변경하는 방식으로 구현한다면 보안상 문제가 생길 것이라 판단
 - 변경할 주소가 다른 학생 것이거나 잘못된 주소이면 안됨
 - 따라서, 기존 주소와 바꿀 주소에 대한 소유권 증명 필요
 - 어떤 주소로 tx를 보냈다는 건 그 지갑 주소의 비밀키를 갖고 있다는 뜻
 - 웹 UX 고려하여 1. 기존 지갑 주소로 proposeAccountChange 호출,
2. 바꿀 지갑으로 confirmAccountChange 호출하여 2번에 걸쳐 변경하도록 구현

Server - DB



| Server - feeDelegation

- 사용자가 kaia 테스트넷의 토큰을 사용하지 않도록 fee delegation(대납) 코드를 수행
1. 웹에서 트랜잭션을 생성 및 서명 (실제 체인에 보내지 않음!!)
 2. 서버로 트랜잭션을 보냄.
 3. 서버에서 서명 후 체인에 보냄. (대신 수수료를 내면서 보내는 과정)

| Polling Server - 사용 이유

- 마일리지는 장학금(돈)과 관련되어 있어 신뢰성이 중요하다.
- 블록체인에 올린 데이터가 확정 블록에 들어가기까지의 시간이 오래걸릴 수 있다 (무작정 올렸다가 블록이 확정되지 않으면 문제!)
- 폴링을 통해 확정 블록으로부터 이벤트를 가져오자!

| Polling Server - 구현

```
const STUDENT_MANAGER_CONTRACT_ADDRESS_EVENT = [  
  .. "DocSubmitted", .. //제출  
  .. "DocApproved", .. //승인(history)  
  .. "DocRejected", .. //거절  
  .. "AccountChanged", .. //계정 변경 완료(hist  
  .. "AdminAdded", .. //관리자 추가  
  .. "StudentRegistered", .. //학생 등록  
  .. "MileageBurned", .. //마일리지 소모(hist  
  .. "MileageMinted", .. //마일리지 발행(hist
```

- listen할 이벤트 정리
- 마지막으로 확인한 블록 저장.
*마지막 확인 블록 부터
체인의 확정 블록까지
확인하며 이벤트 검색

key	value
last_processed_block	188024987

| Polling Server - 구현

- 마지막 확정 블록 ??

소켓을 통해 실시간으로 올라온 블록을 확인하고

(올라온 블록 - n)번째 까지는 확정으로 판단함.

```
polling-1 | New Block: 188024148
polling-1 | 확정 블록 확인 { confirmBlock: 188024145, lastBlock: 188024143 }
polling-1 | New Block: 188024149
polling-1 | Scheduled reconnection attempt 1 in 1000 ms
polling-1 | The websocket connection was closed
polling-1 | New Block: 188024151
polling-1 | New Block: 188024152
polling-1 | 확정 블록 확인 { confirmBlock: 188024149, lastBlock: 188024145 }
```


| Polling Server - 시행착오

- 간헐적으로 서버가 중지되는 상황
 - pm2 모듈을 사용하여 재시작.
(마지막 처리된 블록은 DB에 존재. 로직이 모두 처리되어야만 DB가 업데이트됨)
 - 일시적으로 RPC Endpoint 가 응답하지 않는 것으로 추정

| 프론트엔드

- 전체 UI / UX 재개발
- Suspense - Boundary를 통한 로딩, 에러 선언적 처리
- tanstack-query를 이용한 데이터 캐싱, 네트워크 요청 횟수 최적화
- viem을 이용한 타입 체킹 강화, 다양한 유틸리티 메소드 활용
 - walletClient, publicClient

| 회고

- 이전 프로토타입에 비해 굉장히 많은 기능을 구현했고, 하나의 기능을 완성하기 위해 팀원들과 다양한 경우를 고려하며 설계를 진행한 점이 좋았음. 프로젝트의 완결성을 조금 더 높일 수 있었다고 생각하는데 그렇지 못한 점이 아쉬움.
- 규모가 있는 프로젝트를 진행하며 여러 기능을 구현한 점이 좋았음. 서버는 안정성이 중요하다고 생각하는데 아직 충분히 챙기지 못한 것 같아 아쉬움.
- 백엔드 DB에 바로 저장하는 것 보다 가급적 블록체인에 많은 것을 기록 하는 기획이라 웹, 컨트랙트 어느 쪽이든 생각할 거리가 많아 재밌었음. 막바지에 부수적인 기능들을 급하게 붙이느라 컨트랙트 인터페이스 퀄리티가 떨어진 것이 아쉬움